

CSE3010 • COMPUTER VISION PROJECT

Computer Vision Based Image Tampering Detection

Using Feature Consistency, Segmentation and Geometric Verification

Project Detail	Information
Project Type	Classical Computer Vision / Image Forensics
Primary Task	Blind single-image copy-move tampering detection
Core Techniques	Histogram analysis, SIFT, Lowe's ratio test, Felzenszwalb segmentation, RANSAC
Interface	Command-line interface (CLI), no GUI
Outputs	10 visual artifacts + per-image JSON + evaluation JSON
Testing	39 automated tests; README reports all passing

Student Name: SNEHA CHAUHAN

Registration No.: 24BAI10530

Faculty: Dr.Gaurav Soni

Course: CSE3010 Computer Vision

Academic Year: 2026–27

1. Introduction

Digital image manipulation can be difficult to identify by visual inspection alone. This project develops a classical, inspectable Computer Vision system for blind image-forensics analysis. Instead of using an end-to-end pretrained deep-learning model, the system decomposes the decision into measurable stages: preprocessing, histogram consistency analysis, SIFT feature extraction, self-matching, geometric verification, segmentation, score fusion and reporting.

The repository describes the project as a CSE3010 Computer Vision course project and emphasizes explainability: each number contributing to the final decision can be traced to a specific geometric or statistical computation.

2. Problem Statement

Given a single digital image with no reference original, determine whether it is authentic or manipulated—primarily copy-move forgery—and localize suspicious regions using classical Computer Vision techniques including histogram analysis, SIFT features, segmentation and RANSAC. The core decision does not use a pretrained deep forensic model.

3. Motivation

- Passive image forensics has practical relevance to misinformation, journalism and digital-evidence analysis.
- A classical pipeline directly demonstrates the mathematics behind feature extraction, robust estimation and segmentation.
- Every major stage can produce an inspectable intermediate result, making the system suitable for academic demonstration and debugging.

4. Objectives

- Detect copy-move forgery using self-similarity feature matching.
- Geometrically verify candidate matches with RANSAC to reject coincidental or false matches.
- Localize suspicious regions using segmentation restricted to geometrically verified evidence.
- Produce a transparent weighted-sum tampering score and final decision.
- Generate complete visual and machine-readable reports.
- Support batch evaluation with classification metrics.

5. Functional Requirements

ID	Requirement
FR-01	Accept supported image files including JPG/JPEG, PNG, BMP, TIF and TIFF.
FR-02	Validate, resize, denoise and convert images for downstream processing.
FR-03	Perform block-wise histogram/color consistency analysis.
FR-04	Extract SIFT keypoints and descriptors.
FR-05	Self-match descriptors and filter candidate pairs using Lowe's ratio test.
FR-06	Apply graph-based Felzenszwalb segmentation for candidate localization.
FR-07	Use RANSAC affine transformation estimation for geometric verification.
FR-08	Fuse evidence into a transparent weighted-sum tampering score.
FR-09	Generate 10 visual artifacts for each analyzed image.
FR-10	Generate a per-image machine-readable JSON report.
FR-11	Support labeled evaluation using authentic/ and tampered/ folders.
FR-12	Generate accuracy, precision, recall, F1 and confusion-matrix values.

6. Non-functional Requirements

Property	Requirement
Interpretability	Every major decision stage has traceable, explainable evidence.
Modularity	The src/ package separates preprocessing, features, matching, segmentation, verification, scoring, visualization and evaluation.

Configurability	Tunable parameters are centralized in config.py.
Reproducibility	The same input and configuration should reproduce pipeline outputs.
Auditability	JSON reports preserve intermediate statistics and final decisions.
Testability	Individual modules and the complete pipeline are covered by automated tests.
Usability	The system is executable from the terminal using documented CLI commands.

7. System Architecture

The architecture follows a staged evidence-fusion pipeline. The implemented order is preprocessing → histogram consistency → SIFT extraction → self-matching → RANSAC geometric verification → segmentation using verified inlier pairs → score fusion → decision → reporting.

Stage	Input → Output
1. Validation	Image file → validated image
2. Preprocessing	Validated image → resized/denoised/grayscale data
3. Histogram analysis	Processed image → anomalous block statistics
4. Feature extraction	Gray image → SIFT keypoints/descriptors
5. Self-matching	Descriptors → candidate matched pairs
6. RANSAC verification	Candidate pairs → inliers/outliers + affine transform
7. Segmentation	Verified inlier points → suspicious region localization
8. Score fusion	Evidence → tampering score + decision
9. Reporting	Results → 10 visual artifacts + JSON report

8. Design Diagrams

8.1 Use Case Diagram

Actor	Use Case	Description
Student / Evaluator	Analyze Image	Provide one image and run the complete detection pipeline.
Student / Evaluator	Run Evaluation	Provide a labeled dataset and obtain aggregate metrics.
Student / Evaluator	Inspect Results	Review visual artifacts, console output and JSON reports.
File System	Read / Write	Supply images and store visualizations and reports.

8.2 Workflow Diagram

Step	Process
1	Start → Load image → Validate
2	Resize / denoise / grayscale
3	Histogram & block consistency analysis
4	SIFT feature extraction
5	Self-match using Lowe's ratio test
6	RANSAC geometric verification
7	Segmentation using RANSAC-inlier evidence
8	Weighted tampering score → decision
9	Save 10 visual artifacts + JSON
10	End

8.3 Sequence Diagram

Step	CLI	Preprocessing	CV Pipeline	Reporter
1	analyze(image)	—	—	—
2	—	validate / resize / denoise	—	—
3	—	processed image	extract + match + verify	—
4	—	—	verified regions + score	—
5	—	—	results	save JSON + visual artifacts
6	print summary	—	—	files available

8.4 Class / Component Diagram

Component	Responsibility
main.py / CLI	Argument parsing, mode selection, pipeline orchestration and console reporting.
preprocessing.py	Image validation and preprocessing statistics.
histogram_analysis.py	Block consistency and histogram anomaly analysis.
feature_extraction.py	SIFT keypoint and descriptor extraction.
feature_matching.py	Self-matching and Lowe ratio filtering.
geometric_verification.py	Affine transform estimation and RANSAC inlier/outlier separation.
segmentation.py	Region localization using verified point pairs.
tampering_score.py	Weighted evidence fusion and final decision.
visualization.py	Generation of 10 saved visual artifacts.
evaluation.py	Dataset discovery and aggregate classification metrics.

8.5 ER Diagram (Conditional)

The current implementation is file/JSON based rather than database based, so an ER diagram is not required by the implementation. If results were persisted relationally, the conceptual relationship would be Image → Analysis → Artifact.

Entity	Representative Attributes	Relationship
Image	image_id, path, label	1 : N Analysis
Analysis	score, decision, metrics	N : 1 Image; 1 : N Artifact
Artifact	stage, output_path, metadata	N : 1 Analysis

9. Design Decisions & Rationale

Decision	Rationale
Classical CV baseline	Keeps the core decision inspectable and aligns with the project requirement.
SIFT features	Provides local descriptors for finding repeated image content under common transformations.
Lowe ratio test	Filters ambiguous descriptor matches before geometric reasoning.
RANSAC affine verification	Separates geometrically consistent matches from coincidental matches.
Inlier-only localization	Raw matches were observed to over-flag repetitive textures; localization therefore uses RANSAC inliers.
Felzenszwalb segmentation	Converts verified point evidence into localized candidate regions.
Weighted-sum score	Keeps evidence fusion transparent and configurable.
CLI-only design	Keeps focus on the Computer Vision pipeline and reproducible terminal execution.
Centralized configuration	config.py is the single source of truth for tunable parameters.

10. Implementation Details

The project is implemented in Python 3 using OpenCV, NumPy, SciPy, scikit-image, Matplotlib and pytest. main.py provides single-image detection and labeled-dataset evaluation modes.

The actual pipeline calls preprocessing, histogram analysis, SIFT feature extraction, self-matching, geometric verification, segmentation and score fusion in sequence. Localization receives only RANSAC inlier point pairs.

For single-image mode, the system stores preprocessing, histogram, feature, matching, geometric verification, segmentation and decision information in a per-image JSON report. When enabled, visualization paths for the generated artifacts are also included.

For evaluation mode, labeled images are discovered from authentic/ and tampered/ folders. Predictions are collected into records and aggregate accuracy, precision, recall, F1 and confusion-matrix values are calculated.

11. Screenshots / Results

The supplied README contains a real, unedited sample execution on the synthetic copy-move test image tamp_1.jpg. The documented values are reproduced below.

Pipeline Output	Observed Value
Input image	data/test/tampered/tamp_1.jpg
Image dimensions	800 × 600; processed 800 × 600
Resized	False
Scale factor	1.0000
Histogram anomalies	3 anomalous blocks out of 108
SIFT features	1,030 keypoints
Filtered candidate matches	100
Geometric transform	Found
RANSAC inliers / outliers	43 / 57
Tampering score	0.2330
Final decision	POTENTIALLY TAMPERED
Report	outputs/reports/tamp_1_report.json

Expected Visual Artifacts

Artifact	Purpose
01_original	Original input image.
02_preprocessed	Processed image.
03_histogram	Histogram / consistency visualization.
04_keypoints	SIFT keypoints.
05_matches	Candidate feature matches.
06_segmentation	Segmentation boundaries.
07_candidate_mask	Candidate localization mask.
08_geometric_verification	RANSAC-verified matches.
09_final_heatmap	Final heatmap overlay.
10_summary	Summary panel.

12. Testing Approach

The repository documents 39 automated tests across four test files and reports that all tests passed. Coverage includes invalid and unsupported inputs, corrupted images, very small and grayscale images, low-quality/compressed images, insufficient features, authentic images, copy-move manipulated images and repetitive textures.

Test Category	Coverage
Input validation	Invalid file, unsupported format, corrupted image.
Image robustness	Very small, grayscale, low-quality/heavily-compressed images.
Feature sufficiency	Flat/textureless images with insufficient features.
Functional pipeline	Authentic and copy-move manipulated images.
False-match behavior	Repetitive textures.
Integration	End-to-end pipeline and output generation.

Reported automated result: pytest tests/ -v → 39 passed in 4.95s

13. Evaluation Results

A small synthetic development set containing 3 authentic and 3 copy-move tampered images was evaluated using batch evaluation.

Metric	Value
Images evaluated	6
Accuracy	0.8333
Precision	1.0000
Recall	0.6667
F1-score	0.8000
TP / FP / TN / FN	2 / 0 / 3 / 1

Important: the README explicitly describes this as a 6-image illustrative run, not a benchmark result. A real, larger dataset is required before citing an accuracy figure as a formal benchmark.

14. Challenges Faced

- Blind detection has no trusted original image for direct comparison.
- Feature matching can generate coincidental matches, particularly in repetitive textures.
- Flat or low-texture regions may not provide enough SIFT keypoints.
- Localization from raw matches was observed to over-flag repetitive textures, motivating inlier-only segmentation.
- Threshold calibration on a very small synthetic set is not statistically representative.
- Splicing from a different source image is outside the strongest capability of the copy-move-focused pipeline.

15. Learnings & Key Takeaways

- Classical Computer Vision can be assembled into an end-to-end forensic analysis system without a deep-learning black box.
- SIFT and descriptor matching provide candidate self-similarity evidence, while geometric verification helps reject false matches.
- RANSAC inliers provide a stronger basis for region localization than raw matches.

- Segmentation bridges sparse verified feature evidence and region-level localization.
- Transparent score fusion makes the final decision easier to inspect and debug.
- Automated testing validates behavior across invalid inputs, difficult textures and genuine copy-move examples.
- Visual artifacts plus JSON reports improve reproducibility and evaluation.

16. Future Enhancements

- Evaluate on CASIA v2.0 and CoMoFoD with proper train/dev/test methodology.
- Add IoU-based localization metrics using ground-truth tamper masks.
- Consider an optional lightweight supervised score-fusion layer after obtaining a sufficiently large labeled dataset.
- Add splicing-specific forensic cues such as JPEG double-compression or ELA-style analysis.
- Expand robustness testing for scale, rotation, compression and challenging low-texture regions.
- Provide a GUI/web layer as a future interface without changing the underlying explainable pipeline.

17. References

No.	Reference
1	D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” International Journal of Computer Vision, 2004.
2	M. A. Fischler and R. C. Bolles, “Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography,” Communications of the ACM, 1981.
3	P. F. Felzenszwalb and D. P. Huttenlocher, “Efficient Graph-Based Image Segmentation,” International Journal of Computer Vision, 2004.
4	OpenCV official documentation.
5	scikit-image official documentation.
6	NumPy / SciPy official documentation.
7	Matplotlib official documentation.
8	pytest official documentation.
9	CASIA v2.0 image tampering dataset — recommended external evaluation dataset.
10	CoMoFoD — copy-move forgery dataset with manipulation categories and ground-truth masks.

18. Project Structure

Path	Role
main.py	CLI entry point and pipeline orchestration.
config.py	Central configuration / tunable parameters.
src/preprocessing.py	Preprocessing.

src/histogram_analysis.py	Histogram and block consistency analysis.
src/feature_extraction.py	SIFT feature extraction.
src/feature_matching.py	Self-matching and ratio test.
src/segmentation.py	Candidate region localization.
src/geometric_verification.py	Affine transform + RANSAC.
src/tampering_score.py	Decision score fusion.
src/visualization.py	10 visual artifacts.
src/evaluation.py	Dataset discovery and metrics.
tests/	Automated unit and integration tests.
docs/	Architecture, diagrams and methodology documentation.

Conclusion: The project implements a modular, explainable classical Computer Vision approach to blind copy-move tampering detection. Its key contribution is the traceable pipeline from image input to feature evidence, geometric verification, localized suspicious regions, score fusion and machine-readable reporting.